

Il test reale su dual boot

Le slide una alla volta, ognuna con la sua spiegazione

Ogni pagina ha l'immagine di una slide e sotto la spiegazione. I riquadri grigi sono **esempi con i numeri**; i riquadri gialli segnalano i **punti da correggere o da dire con cautela**; i riquadri verdi le **verifiche** fatte ricontrollando i dati.

Le parole e le unità che tornano sempre

Nodo	Il computer che riceve il pacchetto, esegue la rete neurale e lo inoltra. Nel banco è il core 6.
XDP	Il primo punto del kernel Linux in cui un programma può vedere un pacchetto appena arrivato.
Pipeline	Una delle versioni del programma che esegue la rete neurale (P1, P1.5, P2, P3).
baseline / rxonly	I due programmi di riferimento: la baseline inoltra senza rete neurale, rxonly riceve e basta.
Coda	I 256 posti davanti al nodo dove i pacchetti aspettano. Se è piena, chi arriva è respinto.
Capacità	Quanti pacchetti al secondo il nodo riesce a elaborare al massimo.
Collo di bottiglia	Il pezzo più lento della catena: decide quanto traffico passa.
M/s	Milioni di pacchetti al secondo.
ns, μs, ms	Nanosecondi (miliardesimi di secondo), microsecondi ($1\ \mu\text{s} = 1\ 000\ \text{ns}$), millisecondi ($1\ \text{ms} = 1\ 000\ \mu\text{s}$).
Gbit/s	Miliardi di bit al secondo. Con pacchetti da 64 byte, $1\ \text{Gbit/s} \approx 1,95$ milioni di pacchetti al secondo.
Costo per pacchetto	1 diviso la capacità. 2 milioni di pacchetti al secondo = 500 ns per pacchetto.
Tempo di CPU per pacchetto	Quanto tempo il core del nodo lavora davvero per ogni pacchetto: occupazione del core $\times$ 1 diviso la capacità. Coincide con il costo per pacchetto quando il nodo lavora il 100% del tempo.
Scrittore	Chi mette pacchetti in una coda. Con una scheda di rete vera è la scheda stessa: uno per coda.

TESI · EBPF / XDP

Il test reale su dual boot e la domanda sul bit rate

Come si misura quanti pacchetti regge un nodo, con quali generatori di traffico, e che cosa succede quando il traffico offerto cresce.

Ettore Cantile

settembre 2026

La presentazione risponde a tre domande sul nodo, cioè il computer che riceve un pacchetto, esegue la rete neurale dentro il kernel (con XDP) e decide da quale porta farlo uscire:

1. Quanti pacchetti al secondo regge un nodo?
2. Con quale traffico lo misuriamo (due generatori diversi: pktgen e xdp_gen)?
3. Che cosa succede quando il traffico cresce: dove si perdono i pacchetti? È la domanda del relatore.

Tutte le misure sono state fatte sul portatile con Ubuntu installato direttamente (dual boot), non più dentro la macchina virtuale.

La domanda del relatore

LA DOMANDA DEL RELATORE

Quando il traffico cresce, dove si perde?



Dove si perde dice qual è il **collo di bottiglia**.

Si perde prima del programma

Il pacchetto non è mai entrato: il collo di bottiglia è il **programma**.

Si perde dopo il programma

Il programma lo ha elaborato, ma non è arrivato: il collo di bottiglia è la **trasmissione**.

Per rispondere si contano i pacchetti che il programma **riceve** e quelli che **rilancia**.

La risposta arriva nella parte 5; le parti 1-4 spiegano come si misura

2

Se si manda al nodo sempre più traffico, a un certo punto il nodo non ce la fa più. Davanti al nodo c'è una coda di 256 posti: i pacchetti che il nodo non ha ancora elaborato aspettano lì. Quando la coda è piena, i pacchetti nuovi vengono respinti (persi).

Il relatore chiede di capire **dove** si perdono, perché questo dice chi è il collo di bottiglia:

- se si perdono **prima** del programma (non sono mai entrati), il programma è troppo lento;
- se si perdono **dopo** (il programma li ha elaborati ma non arrivano a destinazione), il problema è la trasmissione in uscita.

Esempio. Il nodo riesce a elaborare 1,9 milioni di pacchetti al secondo e il generatore ne manda 2,8 milioni. Ogni secondo 0,9 milioni di pacchetti non trovano posto e vengono respinti. Se contiamo quanti pacchetti entrano nel programma (1,9 milioni) e quanti escono (sempre 1,9 milioni), vediamo che si perdono tutti prima: il collo di bottiglia è il programma.

Come è organizzata la presentazione

COME È FATTA LA PRESENTAZIONE

Cinque parti, dal banco alla risposta

Ogni parte risponde a una domanda: le prime quattro preparano la risposta alla quinta.

1

Il banco

Che cosa si misura, con quali programmi e come.

Che cosa **misuriamo**?

2

La macchina

Dalla macchina virtuale al dual boot.

Perché fidarsi dei **numeri**?

3

I generatori

pktgen e xdp_gen: due modi di produrre traffico.

Con che **traffico** si misura?

4

I risultati

Quanto regge il nodo, con l'uno e con l'altro.

Quanti **pacchetti** al secondo?

5

La domanda

Che cosa succede quando il traffico cresce.

Dove si **perde**?

IN BREVE

La risposta, in anticipo: si perde **prima** del programma, quindi il collo di bottiglia è **il programma**. E le misure ora si ripetono.

Le parti 2-5 raccontano ciò che è cambiato dal test in macchina virtuale

3

È la mappa delle cinque parti. Le prime quattro servono a fidarsi dei numeri; la quinta risponde alla domanda.

1. **Il banco**: quali programmi si confrontano e come si misura.
2. **La macchina**: perché nella macchina virtuale le misure non erano affidabili e ora sì.
3. **I generatori**: i due modi di produrre traffico.
4. **I risultati**: quanti pacchetti al secondo regge ogni programma.
5. **La domanda**: dove si perde quando il traffico cresce.

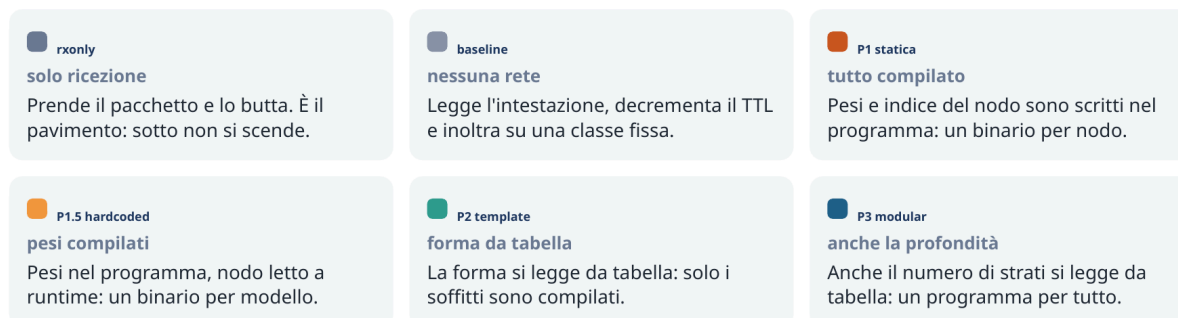
La risposta, anticipata nel riquadro giallo: si perde **prima** del programma, quindi il collo di bottiglia è il programma.

I sei programmi

1 · IL BANCO · I PROGRAMMI

Sei programmi, che cambiano per una cosa sola

Calcolano la stessa rete e prendono la stessa decisione. Cambia quanto sanno già **quando vengono compilati**.



Più informazioni si spostano dalla compilazione all'esecuzione, più il programma è **riutilizzabile**, e più cresce il **costo per pacchetto**.

Cinque pipeline più rxonly, che misura la sola ricezione · i colori dei quadratini tornano in tutti i grafici

4

Le quattro pipeline (P1, P1.5, P2, P3) calcolano la stessa rete neurale e danno la stessa risposta. Cambia solo quanto è "scritto dentro" il programma quando lo si compila, e quanto invece viene letto da tabelle mentre gira.

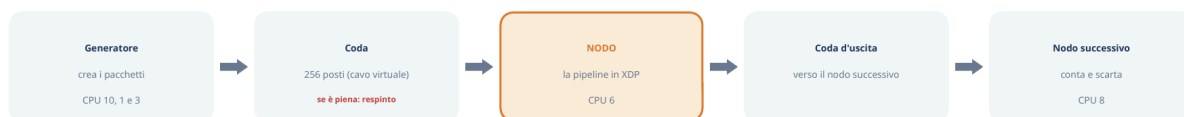
- **rxonly**: riceve il pacchetto e lo butta. Non fa altro. Serve a sapere quanto costa solo ricevere: nessun programma può essere più veloce.
- **baseline**: legge l'intestazione, abbassa il TTL e inoltra, ma senza rete neurale. È il riferimento: quello che le pipeline costano in più rispetto alla baseline è il costo della rete neurale.
- **P1 statica**: pesi e numero del nodo sono scritti nel codice. Esempio: il binario compilato per il nodo 7 non va bene per il nodo 8, che ha bisogno del suo.
- **P1.5**: i pesi sono scritti nel codice, ma il numero del nodo si legge da una tabella. Lo stesso binario va bene per tutti i nodi che usano quel modello.
- **P2**: anche la forma della rete (quanti neuroni) si legge da tabella. Nel codice ci sono solo i soffitti, cioè i valori massimi ammessi. Un solo programma serve molti modelli.
- **P3**: anche il numero di strati si legge da tabella.

Regola generale: più cose si leggono da tabella, più il programma è flessibile, ma più lavoro fa per ogni pacchetto. I colori dei quadratini restano uguali in tutti i grafici.

Il banco di misura

1 · IL BANCO · COME SI MISURA

Il banco misura quanti pacchetti regge il nodo



1

Coda sempre piena

Il generatore offre **più pacchetti** di quanti il nodo riesca a elaborare.

2

Nodo sempre occupato

Il suo core non resta mai senza lavoro: il ritmo a cui elabora è la sua **capacità**.

3

Costo = 1 / capacità

Se elabora 5 milioni di pacchetti al secondo, spende **200 ns** per ciascuno.

QUINDI

Si misurano 300 ms a regime: costo per pacchetto = tempo ÷ pacchetti elaborati. Il generatore sta su altri core, quindi **non entra nella misura**.

Un solo portatile: generatore, nodo e nodo successivo ciascuno sul proprio core fisico

5

Tutto gira sullo stesso portatile, ma ogni pezzo ha il suo core (processore) dedicato:

- il **generatore** (CPU 10, 1, 3) crea i pacchetti;
- la **coda** da 256 posti è quella del cavo virtuale che collega generatore e nodo;
- il **nodo** (CPU 6) esegue la pipeline: è l'unica cosa che vogliamo misurare;
- il **nodo successivo** (CPU 8) conta i pacchetti che arrivano e li butta.

Come si misura il costo. Si manda più traffico di quanto il nodo possa elaborare, così la coda è sempre piena e il nodo non si ferma mai. A quel punto il ritmo a cui elabora è la sua capacità, e il tempo speso per ogni pacchetto è 1 diviso la capacità.

Esempio. In 300 millisecondi il nodo elabora 900 000 pacchetti. Sono 3 milioni al secondo, quindi spende 1 / 3 000 000 di secondo per pacchetto, cioè circa 333 ns.

Perché mandare più traffico del necessario? Se il generatore mandasse meno pacchetti di quanti il nodo ne regge, il nodo starebbe fermo ogni tanto e misureremmo il ritmo del generatore, non quello del nodo.

Aggiunto il 30 settembre. Il banco misura anche quanto è occupato il core del nodo, nella stessa finestra in cui conta i pacchetti. Con pktgen è occupato al 100%, e 1 diviso la capacità è davvero il suo costo. Con xdp_gen a un thread il nodo a tratti aspetta (slide 13): lì il costo si calcola come **tempo di CPU per pacchetto**, cioè occupazione per 1 diviso la capacità.

Il problema della macchina virtuale

2 · MACCHINA VIRTUALE E DUAL BOOT · IL PROBLEMA

Nella macchina virtuale il nodo ogni tanto si fermava

Quanto può stare fermo il nodo

prima che la coda da 256 posti trabocchi

Riempire la coda (1,5 M pacchetti/s)

Risveglio da sonno profondo

Pausa di un core virtuale

la coda trabocca

170 μ s310 μ s

alcuni millisecondi

Due strade, entrambe cattive

Generatore e nodo su core separati

Il nodo perdeva **fra l'8 e il 70%** dei pacchetti a qualunque ritmo, anche lentissimo.

Generatore e nodo sullo stesso core

Non si perdeva niente, ma il core faceva il lavoro di tutti e due: il collo di bottiglia era sempre **il generatore**.

QUINDI

La perdita misurava **le assenze del core**, non il programma: dentro la macchina virtuale un core può fermarsi per millisecondi, e non si vede né si può impedire.

6

Fino al 26 settembre il test girava dentro VirtualBox. Un "core" della macchina virtuale non è un core vero: il sistema che ospita la macchina virtuale può fermarlo quando vuole, anche per qualche millisecondo, e dall'interno non lo si vede.

Quanto tempo può stare fermo il nodo? Dipende da quanto ci mette la coda a riempirsi. A 1,5 milioni di pacchetti al secondo ne arriva uno ogni $0,67 \mu\text{s}$, quindi 256 posti si riempiono in $256 \times 0,67 \approx 170 \mu\text{s}$. Qualunque pausa più lunga fa perdere pacchetti.

Esempio. Il core virtuale viene fermato per 2 millisecondi. In quel tempo arrivano $2 \text{ ms} \times 1,5 \text{ milioni/s} = 3\,000$ pacchetti, ma la coda ne contiene 256: circa 2\,750 vanno persi. E il programma non c'entra niente: poteva essere velocissimo.

Si poteva scegliere solo fra due strade sbagliate:

- generatore e nodo su core separati: perdite dall'8 al 70% anche a traffico bassissimo, cioè si misuravano le pause del core;
- tutto sullo stesso core: niente perdite, ma il core faceva il lavoro del generatore e del nodo insieme, e il limite era sempre il generatore.

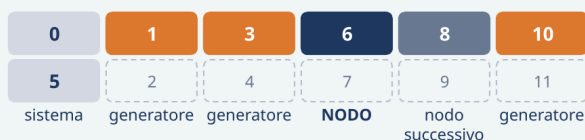
La soluzione: il dual boot

2 · MACCHINA VIRTUALE E DUAL BOOT · LA SOLUZIONE

Sul dual boot ogni core ha un ruolo fisso

La macchina: un processore ibrido

Intel Core Ultra 7 155H, 22 CPU: ognuna ha il suo ruolo



E-core 12-19 e LP E-core 20-21: sistema

In tratteggio: il thread gemello di ogni core, lasciato a riposo.

1 Frequenza fissa e misurata

3,5 GHz richiesti, 3 494 MHz misurati dai contatori del processore durante ogni misura.

2 Niente sonno profondo

Un core che dorme impiega 310 μ s a svegliarsi, più dei 170 μ s che la coda concede.

3 Il resto del sistema altrove

Desktop, servizi, interruzioni e thread del kernel stanno sulle altre CPU.

4 Un monitor sempre acceso

Temperatura, throttling e frequenza vera: una finestra fuori condizione si conta, non si scarta.

QUINDI

Il core del nodo **non dorme**, **non cambia frequenza** e **nessuno lo interrompe**: si misura il programma, non la macchina.

7

Dal 27 settembre Ubuntu gira direttamente sul portatile. Lo script `host_conditions.py` prepara la macchina prima di ogni misura e la rimette com'era alla fine:

- **ogni ruolo ha un core fisico suo** (schema a sinistra). Il processore ha core con due "thread gemelli" (SMT): il gemello di ogni core usato resta spento, perché due thread sullo stesso core si dividono le risorse e si rallentano a vicenda;
- **frequenza fissa**: 3,5 GHz, e si verifica con i contatori del processore che sia davvero quella (3 494 MHz misurati);
- **niente sonno profondo**: un core che si addormenta ci mette 310 μ s a svegliarsi, più dei 170 μ s che la coda concede (slide 6);
- **tutto il resto altrove**: desktop, servizi e interruzioni girano sulle altre CPU;
- **un monitor** controlla temperatura e frequenza: se una misura avviene in condizioni sbagliate viene segnalata.

Risultato: il core del nodo non si ferma, non cambia velocità e nessuno lo disturba. Si misura il programma, non la macchina.

Prima e dopo

2 · MACCHINA VIRTUALE E DUAL BOOT · PRIMA E DOPO

Con il dual boot le misure si ripetono

Macchina virtuale		Dual boot		
Pipeline	costo	variazione	costo	variazione
P1 statica	~82 ns	160%	38 ns	0,5%
P1.5	~116 ns	104%	53 ns	0,6%
P2	~430 ns	19%	226 ns	0,5%
P3	~670 ns	15%	364 ns	0,3%

Costo sopra la baseline, per pacchetto. Le cifre assolute non si confrontano (frequenza e metodo erano diversi): si confronta la variazione.

160% → 0,5%

variazione del costo di P1 fra le esecuzioni

1-2% → 0

pacchetti respinti sotto capacità, anche per il solo contatore

58% · 0,2% · 7,7%

perdita dello stesso punto in tre giri (macchina virtuale). Sul dual boot: entro lo 0,4-2%

QUINDI

Il guadagno vero non è la velocità, è la **ripetibilità**: differenze che prima sparivano nel rumore, come P1 contro P1.5 o scartare contro inoltrare, ora si misurano.

Macchina virtuale: 18 settembre, 512 byte · dual boot: 28 settembre, core separati, 64 byte · variazione fra le esecuzioni

8

Da quale test vengono i numeri. La tabella viene dal test di throughput con traffico vero e generatore pktgen (lo stesso della slide 12). "Costo" è quanti nanosecondi per pacchetto una pipeline spende in più della baseline, cioè il costo della rete neurale. "Variazione" dice quanto cambia il risultato ripetendo la stessa misura.

Esempio. Nella macchina virtuale P1 una volta costava 164 ns e un'altra 32 ns: variazione enorme. Con numeri così non si può dire se P1 sia più veloce di P1.5. Sul dual boot le ripetizioni danno quasi sempre lo stesso valore, e la differenza di circa 15 ns fra P1 e P1.5 si vede.

Gli altri due numeri a destra:

- **1-2% → 0**: con un traffico più basso della capacità, nella macchina virtuale si perdeva comunque l'1-2% dei pacchetti (per le pause del core); sul dual boot nessuno;
- **58% · 0,2% · 7,7%**: nella macchina virtuale lo stesso punto del test sul bit rate, ripetuto tre volte, dava tre perdite completamente diverse.

Attenzione. Le due colonne "variazione" non misurano esattamente la stessa cosa: quella della macchina virtuale riguarda il costo sopra la baseline, quella del dual boot riguarda i pacchetti al secondo di ogni pipeline. Misurata come per la macchina virtuale, la variazione sul dual boot fra sessioni diverse è circa 1,6 / 3,2 / 2,5 / 3,4% (P1 / P1.5 / P2 / P3). Il miglioramento resta enorme (P1: da 160% a circa 2%), ma lo 0,5% della slide lo fa sembrare più grande di quanto sia.

Il primo generatore: pktgen

3 · I DUE GENERATORI DI TRAFFICO · PKTGEN

pktgen produce i pacchetti come li produce Linux



≈ **5,7 M/s** pacchetti al secondo offerti da tre thread su tre core

≈ **4,6 M/s** quanti ne accetta la coda del nodo: lo misura rxonly

QUINDI

La copia è una proprietà del kernel, **non della macchina virtuale**: il dual boot non la toglie. Su una scheda di rete con XDP nativo non esisterebbe.

La baseline costa 302 ns per pacchetto con pktgen · M/s = milioni di pacchetti al secondo

9

pktgen è un generatore di traffico che sta dentro il kernel Linux. Crea i pacchetti come li crea il resto del sistema: dentro una struttura chiamata **skb**, che lascia 64 byte liberi davanti ai dati.

XDP però vuole 256 byte liberi davanti al pacchetto (servono a un programma che volesse aggiungere intestazioni). Allora il cavo virtuale, prima di consegnare il pacchetto al nodo, lo **copia** in una pagina nuova con lo spazio giusto. Questa copia la fa il core del nodo, quindi finisce nel tempo che misuriamo.

Esempio. Con pktgen P2 costa 529 ns per pacchetto; con l'altro generatore, che non richiede la copia, 293 ns di CPU. Circa 230 ns del tempo misurato con pktgen sono quindi trasporto e copia, non lavoro della pipeline, e sono gli stessi per tutte le pipeline (slide 15).

Il passaggio al dual boot non toglie la copia: è un comportamento del kernel, non della macchina virtuale. Su una scheda di rete vera con XDP nativo non ci sarebbe. Con tre thread pktgen offre circa 5,7 milioni di pacchetti al secondo, e la coda del nodo ne accetta circa 4,6 (quanto regge rxonly).

Il secondo generatore: xdp_gen

3 · I DUE GENERATORI DI TRAFFICO · XDP_GEN

xdp_gen consegna pacchetti già nel formato di XDP

**12-15**milioni di pacchetti al secondo
offerti da un solo thread**293 ns**P2 con xdp_gen, contro 529 ns
con pktgen**QUINDI**

Il costo del generatore non passa più dal nodo. Si usa **un thread per coda**: più thread nella stessa coda si intralciano e rallentano anche il nodo (slide 13).

Frame grezzi creati con BPF_PROG_TEST_RUN in modalità live frames · script ipa/test/xdp_gen.py

10

xdp_gen (script `ipa/test/xdp_gen.py`) usa una funzione del kernel, `BPF_PROG_TEST_RUN` in modalità **live frames**: il kernel prepara dei pacchetti già nel formato che XDP si aspetta (con i 256 byte liberi) e li spedisce sul cavo virtuale. Il pacchetto arriva al nodo così com'è: niente skb e niente copia, come arriverebbe da una scheda di rete vera.

Esempio. Lo stesso programma, P2, costa 529 ns con pktgen e 293 ns di CPU con xdp_gen. Il programma non è cambiato: è cambiato solo il modo in cui il pacchetto arriva.

Un solo thread di xdp_gen offre 12-15 milioni di pacchetti al secondo, e si usa così: **un thread per ogni coda del nodo**. Con più thread nella stessa coda i thread si intralciano e rallentano anche il nodo (slide 13).

Verificato il 30 settembre. Con tre thread il nodo si fermava intorno a 7 milioni al secondo, e sembrava il tetto del generatore. Non lo era: con **un thread** la baseline elabora 12,4 milioni e rxonly 14,9. Il limite era la coda d'ingresso condivisa da tre scrittori (slide 13).

I due generatori a confronto

3 · I DUE GENERATORI DI TRAFFICO · A CONFRONTO

Due generatori, per due scopi diversi

Stesso nodo, stessi programmi: cambia solo **come arriva il pacchetto**.

Confronto	pktgen	xdp_gen
Come crea il pacchetto	una skb, come lo stack di Linux	un frame grezzo, già nel formato di XDP
Copia prima del nodo	sì, a ogni pacchetto	no
Pacchetti offerti	≈ 5,7 M/s (il nodo ne accetta ≈ 4,6)	≈ 12–15 M/s (un thread)
Chi arriva al tetto	nessuna pipeline: tutte e cinque saturano sotto	baseline e rxonly; P1, P1.5, P2 e P3 saturano
Serve per	confrontare tutte le pipeline fra loro	misurare il costo di P1–P3 senza la copia

QUINDI Si usano **insieme**: ognuno confronta le pipeline al suo interno, e le cifre dei due generatori non si mescolano.

Il tetto di pktgen è il tetto di ricezione del nodo (rxonly, 4,6 M/s) · M/s = milioni di pacchetti al secondo

11

La tabella riassume le due slide precedenti. La riga importante è "Serve per":

- **pktgen** è più lento, quindi tutte le pipeline arrivano al loro limite prima di lui. Si possono confrontare tutte e cinque fra loro, e anche rxonly satura. È la misura principale.
- **xdp_gen** toglie il costo della copia, quindi dà il costo più vicino a una scheda vera. Con un thread per coda P1, P1.5, P2 e P3 saturano; baseline e rxonly no, e per loro il valore è indicativo. Con xdp_gen si citano quindi P1–P3.

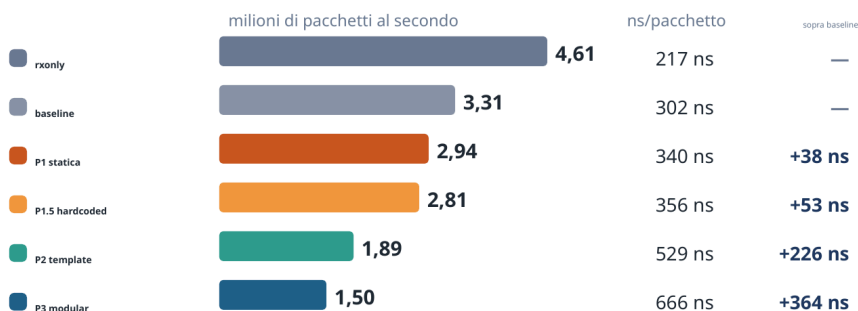
Esempio d'uso. Per dire "P1 è più veloce di P1.5 di circa 15 ns" si usa pktgen. Per dire "P2 costa circa 293 ns di CPU per pacchetto senza il sovraccarico della copia" si usa xdp_gen.

Le cifre dei due generatori non si mescolano: ognuno si usa per confrontare le pipeline al suo interno. **Sopra la baseline**, però, danno lo stesso costo della rete neurale (slide 15).

Risultati con pktgen

4 - I RISULTATI SUL DUAL BOOT - PKTGEN

Con pktgen tutte le pipeline saturano e si ordinano



Sotto capacità, zero perdite

Con 1,37 M/s offerti (il 91% di quanto regge la più lenta) nessuna pipeline respinge un pacchetto: perdita totale $\leq 0,02\%$.

P1 e P1.5 si separano

2,94 contro 2,81 M/s, con variazioni dello 0,5 e 0,6%: congelare il nodo vale circa 15 ns per pacchetto.

QUINDI

Sopra la baseline, la rete neurale costa ~40–50 ns con i pesi compilati (P1, P1.5), ~220 ns con la forma letta da tabella (P2), ~370 ns con anche la profondità (P3).

3 giri × 3 ripetizioni, 64 byte, 28 settembre · variazione fra i giri 0,2–0,9% · DUT a 3 494 MHz, nessun throttling

12

Per ogni programma: quanti milioni di pacchetti al secondo regge (la barra), quanti nanosecondi spende per pacchetto, e quanto costa in più della baseline, cioè il costo della rete neurale.

Esempio di calcolo. P2 regge 1,89 milioni di pacchetti al secondo, quindi spende $1 / 1\,890\,000 \text{ s} \approx 529 \text{ ns}$ per pacchetto. La baseline ne spende 302. La differenza, $529 - 302 \approx 226 \text{ ns}$, è il tempo che P2 dedica alla rete neurale: ricevere, leggere l'intestazione e inoltrare lo fanno tutte e due allo stesso modo.

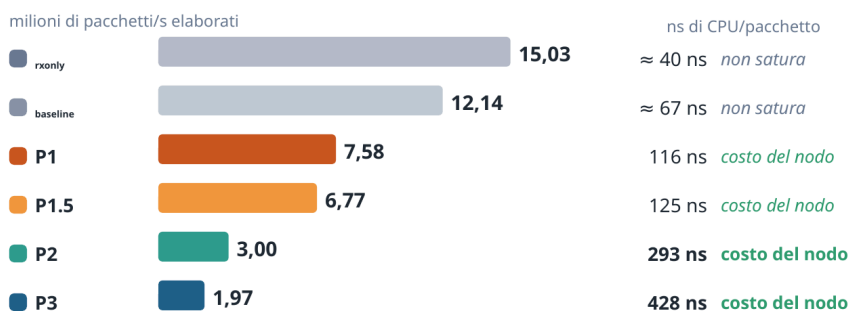
- **Pesi scritti nel codice (P1, P1.5):** la rete costa 40–50 ns. **Letti da tabella (P2):** circa 220 ns. **Anche la profondità da tabella (P3):** circa 370 ns.
- **Sotto capacità, zero perdite:** mandando a tutte 1,37 milioni di pacchetti al secondo (meno di quanto regge la più lenta) nessuna perde pacchetti.
- **P1 contro P1.5:** 2,94 contro 2,81 milioni al secondo. Congelare il numero del nodo nel codice fa risparmiare circa 15 ns per pacchetto: poco, ma misurabile, perché le ripetizioni variano meno dell'1%.

Verificato il 30 settembre. Con pktgen il core del nodo è occupato al 100%, quindi questi sono tempi di CPU veri. I tre thread aggiungono circa 17 ns di contesa sulla coda, uguali per tutte (baseline 278 ns con un thread contro 296 con tre): i costi **sopra la baseline** non cambiano.

Risultati con xdp_gen

4 - I RISULTATI SUL DUAL BOOT - XDP_GEN

Con xdp_gen, un thread per coda: il costo del nodo



Il tetto era la coda

Con tre thread nella stessa coda il nodo si fermava a ~7 M/s. Con un thread per coda la baseline elabora 12 M/s.

QUINDI

P1, P1.5, P2 e P3 sono **costi del nodo**: 116, 125, 293 e 428 ns di CPU. Baseline e rxonly, con un thread, non saturano: i loro valori sono indicativi.

1 thread, 1 core, uscita separata - 3 giri, 64 byte - costo = tempo di CPU del nodo per pacchetto - a 512 e 1 514 byte P2 e P3 restano entro il 3%

13

Un thread di generatore, un core del nodo, uscita su un core suo: **uno scrittore per ogni coda**, come con una scheda di rete vera. La colonna a destra è il **tempo di CPU** che il nodo spende per ogni pacchetto.

- **P1, P1.5, P2 e P3 saturano** (una parte dei pacchetti viene respinta): 116, 125, 293 e 428 ns sono costi del nodo.
- **Baseline e rxonly non saturano**: un thread offre 12–15 milioni al secondo e il nodo li prende tutti. Il loro tempo di CPU (≈ 67 e ≈ 40 ns) è misurato con pochi pacchetti per giro di ricezione, ed è indicativo.

Come si è capito che il tetto era la coda. Con tre thread tutti i programmi leggeri si fermavano intorno a 7 milioni al secondo. Togliendo thread il nodo elabora **di più**: la baseline 7,2 milioni con tre thread, 7,3 con due, 12,4 con uno. Con due core (due code): due thread, uno per coda, danno 13,0 milioni; tre thread, due sulla stessa coda, 7,6. I thread che scrivono nella stessa coda si passano un lucchetto e le stesse righe di cache, e il core del nodo che la svuota ne subisce il rallentamento. P2 e P3, il cui limite è il programma, non ne risentono (+6% e +1%).

Perché il tempo di CPU. Un thread solo spedisce a raffiche: la raffica riempie la coda e il resto viene respinto, poi fra una raffica e l'altra il nodo svuota la coda e aspetta. Il core del nodo lavora così l'84–88% del tempo, anche mentre respinge il 70–80% dei pacchetti. 1 diviso i pacchetti elaborati conterebbe anche l'attesa: P2 risulterebbe 334 ns invece di 293.

Verificato il 30 settembre: scartare costa più che inoltrare, circa 50 ns: nella misura per classe la classe DROP costa 173 ns su P1 contro 117, 480 su P3 contro 427. Quando il pacchetto viene scartato la sua memoria si libera sul core del nodo; quando viene inoltrato, su quello d'uscita. La vecchia osservazione "la baseline supera rxonly" invece non si ripete: nelle misure pulite la baseline non è più veloce di rxonly. Era un effetto della coda contesa.

Con xdp_gen si contano i pacchetti elaborati

4 · I RISULTATI SUL DUAL BOOT · XDP_GEN

Con xdp_gen si contano i pacchetti elaborati

Milioni di pacchetti al secondo, con xdp_gen



Due core, le stesse code d'uscita

Con due core sul nodo, tutti e due inoltrano nelle stesse code verso il nodo successivo: due scrittori per coda. Le code si intasano e metà dei pacchetti della baseline si perde dopo il nodo, con il core d'uscita occupato solo al 77%.

Un esempio: baseline e P2

La baseline elabora 13,1 M/s ma ne arrivano 4,3; P2 ne elabora 6,5 e ne arrivano 6,0. Contando gli arrivi la baseline sembrerebbe più lenta di P2, che fa molto più lavoro.

QUINDI

Il controllo di validità (la baseline deve essere la più veloce) conta i pacchetti **elaborati**: la stessa grandezza con cui si calcola il costo.

xdp_gen, 2 core sul nodo, un thread per coda, uscita separata · 64 byte, 30 settembre · arrivati = nodo successivo

14

Ci sono due modi di contare: i pacchetti **elaborati** dal nodo (le decisioni della pipeline) e quelli **arrivati** al nodo successivo. Di solito coincidono: con un thread per coda e un core sul nodo il nodo successivo riceve 12 milioni al secondo senza perderne. Si separano quando l'uscita non tiene il passo.

Qui il nodo ha **due core**, con un thread di generatore per coda. Tutti e due i core inoltrano nelle **stesse** code verso il nodo successivo: due scrittori per coda, come i tre thread nella coda d'ingresso della slide 13. Le code d'uscita si intasano, il redirect fallisce e il pacchetto si perde dopo il nodo, anche se il core d'uscita è occupato solo al 77%.

Esempio. La baseline elabora 13,05 milioni di pacchetti al secondo, ma al nodo successivo ne arrivano 4,26. P2 ne elabora 6,54 e ne arrivano 5,98. Se contassimo gli arrivi, la baseline sembrerebbe più lenta di P2, che fa molto più lavoro. P3 (4,67) è sotto il limite dell'uscita: le due barre sono uguali.

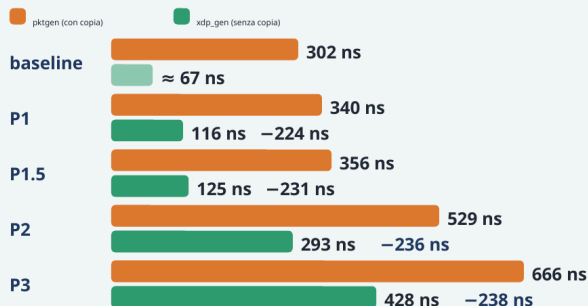
Per questo il banco usa sempre i pacchetti elaborati, che sono anche quelli con cui calcola il costo. E per misurare il costo del nodo si usa un solo core (slide 13), dove l'uscita non perde niente.

La differenza fra i due generatori

4 · I RISULTATI SUL DUAL BOOT · LE DIFFERENZE

Con xdp_gen lo stesso programma costa ~230 ns in meno

Nanosecondi per pacchetto, stesso programma



Con pktgen si citano tutte e cinque

Costo sopra la baseline: +38, +53, +226 e +364 ns per P1, P1.5, P2 e P3.

Con xdp_gen si citano P1, P1.5, P2 e P3

116, 125, 293 e 428 ns di CPU, un thread per coda. Baseline ≈ 67 ns: indicativa.

Da dove vengono i ~230 ns

Dalla copia della skb e dal percorso dello stack: il costo del generatore, pagato sul core del nodo.

QUINDI

I due generatori **non si confrontano** fra loro: ognuno confronta le pipeline al suo interno. La differenza è il costo del percorso skb.

pktgen del 28 settembre, xdp_gen del 30 (tempo di CPU) · P1: $340 - 116 = 224$ · P1.5: $356 - 125 = 231$ · P2: $529 - 293 = 236$ · P3: $666 - 428 = 238$ ns

15

Lo stesso programma misurato con i due generatori. La barra arancione (pktgen) è sempre più lunga di quella verde (xdp_gen): la differenza è il costo della copia della skb e del percorso nel kernel, pagato dal core del nodo.

Esempio. P2: 529 ns con pktgen, 293 ns con xdp_gen, differenza 236 ns. P3: 666 contro 428, differenza 238 ns. P1: 224, P1.5: 231. La differenza è praticamente uguale per tutte le pipeline: è un costo fisso del trasporto, che non dipende dalla rete neurale.

Per la baseline la barra verde è chiara: con un thread non satura, e il suo valore (≈ 67 ns) è indicativo.

Conclusione: i due generatori non si confrontano fra loro. Ciascuno serve a confrontare le pipeline fra di loro, con lo stesso trasporto per tutte. Le cifre di pktgen sono del 28 settembre, quelle di xdp_gen del 30 (tempo di CPU, un thread per coda). La cifra dei circa 230 ns è un calcolo fatto da questi numeri.

Come si risponde alla domanda: due programmi in catena

5 · LA DOMANDA DEL RELATORE · COME SI MISURA

Due programmi sullo stesso aggancio, tre conteggi

Un'interfaccia porta un solo programma XDP: i due si mettono **in catena**, con un salto.



inviati – ricevuti

= persi **prima** del programma
respinti dalla coda d'ingresso

ricevuti – inoltrati

= persi **dopo** il programma
o decisioni del modello (scarta)

rxonly = il solo programma 1

Dice quanto regge la sola ricezione allo
stesso ritmo. Il programma 1 costa fino a
+5 ns: dentro il rumore.

Il salto è la tail call che le pipeline già usano · costo del programma 1 misurato con le statistiche del kernel

16

Per sapere dove si perdono i pacchetti servono tre contatori, messi in tre punti diversi:

1. **inviati**: li conta il generatore;
2. **ricevuti**: li conta un piccolo programma messo davanti alla pipeline, che fa solo `ing_count += 1` e poi passa il pacchetto alla pipeline;
3. **inoltrati**: li conta il nodo successivo.

Un'interfaccia può avere un solo programma XDP. Per averne due, il primo (il contatore) salta al secondo (la pipeline) con una **tail call**, lo stesso meccanismo che le pipeline usano già al loro interno. Il contatore costa pochissimo: fino a 5 ns, dentro il rumore di misura.

Esempio. In una finestra di misura: inviati 1 000 000, ricevuti 800 000, inoltrati 800 000. Allora 200 000 pacchetti si sono persi prima del programma (respinti dalla coda) e zero dopo.

rxonly è il contatore da solo, senza pipeline dietro: dice quanto regge la sola ricezione.

Come si leggono i tre conteggi

5 · LA DOMANDA DEL RELATORE · COME SI LEGGE

Dove mancano i pacchetti indica il collo di bottiglia

Un programma XDP non ha una coda sua: se è lento, si riempie la coda che sta **davanti** a lui.

SI OSSERVA

inviati > ricevuti

mancano pacchetti prima di XDP



VUOL DIRE

La coda d'ingresso è piena: il collo di bottiglia è **il programma** (o la sola ricezione: lo separa rxonly).

SI OSSERVA

ricevuti > inoltrati

al nodo successivo ne arrivano meno: si guarda che cosa aveva deciso il programma



SE AVEVA DECISO DI INOLTARE

Il pacchetto non è arrivato: il collo di bottiglia è **la trasmissione**.

SE AVEVA DECISO DI SCARTARE (O NON USATA)

Non è una perdita: è **una decisione** della rete neurale.

QUINDI

Tre conteggi più le decisioni del programma separano **tre casi** che altrimenti sembrerebbero uguali.

Inviati: generatore · ricevuti: programma 1 · inoltrati: nodo successivo · decisioni: contatori della pipeline (HIT, MISS, DROP)

17

Un programma XDP non ha una coda sua: elabora un pacchetto alla volta, dall'inizio alla fine. Se è lento, si riempie la coda che sta davanti a lui. Per questo i tre conteggi dicono chi è il collo di bottiglia:

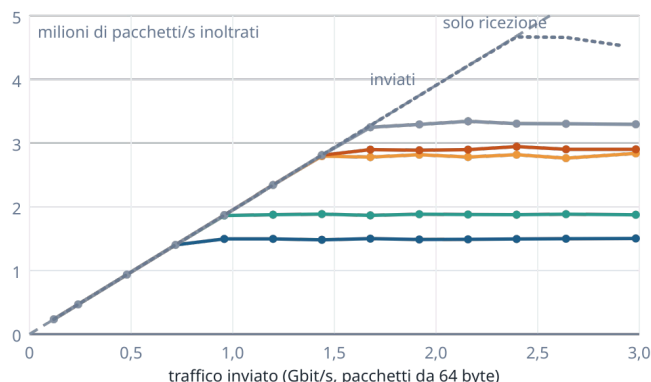
- **inviati > ricevuti**: mancano pacchetti prima di XDP, la coda d'ingresso era piena. Il collo di bottiglia è il programma (oppure la sola ricezione: lo si capisce confrontando con rxonly, slide 21).
- **ricevuti > inoltrati**: il programma li ha presi ma non sono arrivati. Qui si guarda cosa aveva deciso il programma (i suoi contatori HIT, MISS, DROP): se aveva deciso di inoltrarli, si sono persi in uscita e il collo di bottiglia è la trasmissione; se aveva deciso di scartarli, non è una perdita, è la rete neurale che ha scelto così.

Esempio. Ricevuti 100, inoltrati 90. Se il programma ha deciso "scarta" per 10 pacchetti, va tutto bene. Se invece aveva deciso "inoltre" per tutti e 100, 10 si sono persi in uscita.

Le curve: pacchetti inoltrati al crescere del traffico

5 · LA DOMANDA DEL RELATORE · PACCHETTI INOLTRATI

Ogni curva sale con il traffico e poi si ferma



Dove si ferma ogni pipeline

pipeline	pulita fino a (Gbit/s)	perde da (Gbit/s)	inoltrato max (M pacchetti/s)
baseline	1,44	1,68	3,34
P1	1,20	1,44	2,94
P1.5	1,20	1,44	2,84
P2	0,72	0,96	1,89
P3	0,72	0,96	1,50

Sono gradini della scala (~0,24 Gbit/s), non soglie esatte. A 64 byte, 1 Gbit/s $\approx$ 1,95 milioni di pacchetti al secondo.

QUINDI

Ogni curva si ferma alla **capacità della sua pipeline**: 3,3 · 2,9 · 2,8 · 1,9 · 1,5 milioni di pacchetti al secondo, la stessa cifra del test di saturazione.

13 valori di traffico, 5 giri ciascuno, 64 byte, 390 finestre · DUT a 3 494 MHz, nessun throttling

18

Come si legge il grafico. In orizzontale c'è il traffico inviato (in Gbit/s), in verticale i pacchetti che il nodo riesce a inoltrare (milioni al secondo). La linea tratteggiata grigia è "tutto quello che si invia": finché una curva sta sopra la diagonale, il nodo inoltra tutto. Quando la curva diventa piatta, il nodo è arrivato alla sua capacità e il resto si perde.

Da Gbit/s a pacchetti. Un pacchetto da 64 byte è 512 bit. Quindi $1 \text{ Gbit/s} = 1\,000\,000\,000 / 512 \approx 1,95$ milioni di pacchetti al secondo. P2 si ferma a 1,89 milioni di pacchetti al secondo, cioè $1,89 \times 0,512 \approx 0,97$ Gbit/s.

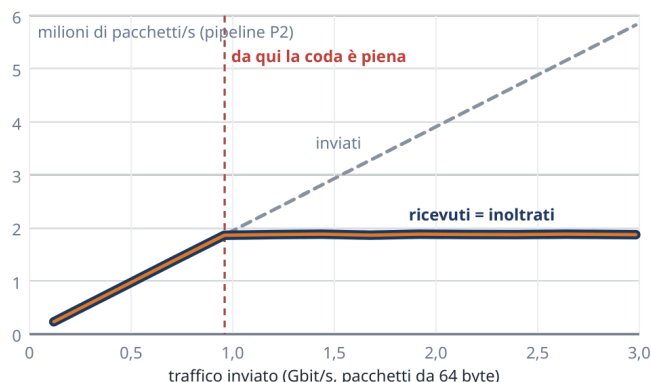
Le cinque pipeline si fermano esattamente alla capacità trovata nel test di saturazione (slide 12): due test diversi danno lo stesso numero. La curva tratteggiata in alto (rxonly) si ferma molto più in là, a circa 2,4 Gbit/s.

Le colonne della tabella. Il traffico si prova solo a 13 valori, distanti circa 0,24 Gbit/s. "Pulita fino a" è l'ultimo valore provato senza perdite, "perde da" il primo con perdite: la soglia vera sta fra i due. Esempio: P2 è pulita a 0,72 e perde a 0,96, e la sua soglia vera è 0,97 (proprio sul gradino). P3 ha gli stessi gradini di P2: la differenza fra le due si legge nell'ultima colonna (1,89 contro 1,50).

Ricevuti e inoltrati coincidono

5 · LA DOMANDA DEL RELATORE · RICEVUTI E INOLTRATI

Il programma rilancia tutto ciò che riceve



1

Sotto il limite, le curve coincidono

Tutto ciò che si invia viene ricevuto e inoltrato.

2

Sopra il limite, salgono solo gli inviati

Ricevuti e inoltrati si fermano insieme, a 1,9 milioni al secondo.

3

Ciò che manca non è mai entrato

Il programma non perde niente di quello che prende.

QUINDI

Ricevuti e inoltrati restano sovrapposti in tutte e cinque le pipeline: dopo XDP si perde al massimo lo 0,02% degli inviati.

Esempio: P2, mediana di 5 giri · linea scura = ricevuti da XDP, arancione = inoltrati; coincidono

19

Il grafico mostra una sola pipeline, P2, con tre curve: inviati (tratteggiata), ricevuti dal programma (linea scura) e inoltrati (arancione). Ricevuti e inoltrati sono una sopra l'altra per tutto il grafico: tutto quello che il programma riceve, lo rilancia.

Esempio con i numeri veri. A 1,44 Gbit/s si inviano 2,81 milioni di pacchetti al secondo. P2 ne riceve 1,89 e ne inoltra 1,89. I 0,92 milioni mancanti si sono persi tutti prima del programma, alla coda d'ingresso.

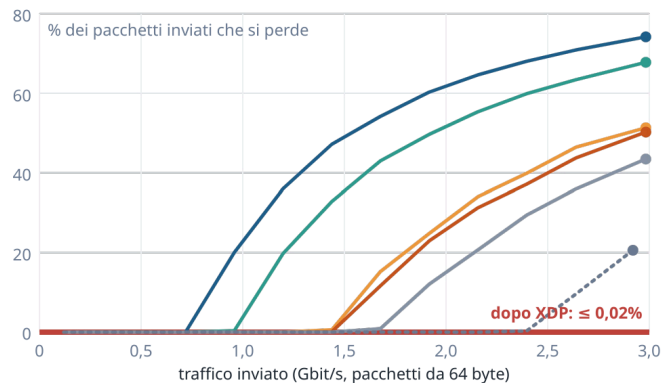
La linea rossa verticale segna il punto da cui la coda è piena: a sinistra le tre curve coincidono, a destra sale solo quella degli inviati.

Sullo "0,02%". Non è una perdita vera: è rumore di misura. Il banco legge i contatori uno dopo l'altro, non tutti nello stesso istante, e in quel breve intervallo passano alcune centinaia di pacchetti. Su 325 misure il valore è negativo nel 47% dei casi (una perdita vera non può essere negativa) e in media è zero. Più corretto dire: dopo il programma la perdita è zero, entro il rumore di misura.

La perdita è tutta prima del programma

5 · LA DOMANDA DEL RELATORE · LA PERDITA

La perdita è tutta prima del programma



Perdita a circa 3 Gbit/s

prima del programma, % degli inviati

P3	74%
P2	68%
P1.5	51%
P1	50%
baseline	43%
rxonly (solo ricezione)	21%

Dopo il programma: ≤ 0,02%

in tutte e cinque le pipeline, a ogni traffico

QUINDI

Se si perde all'ingresso il collo di bottiglia è il programma: qui la perdita è **tutta all'ingresso**, quindi il collo di bottiglia non è la trasmissione.

Perdita = (inviati - inoltrati) / inviati · mediana di 5 giri · a circa 3 Gbit/s si inviano 5,8 M pacchetti/s (5,7 per rxonly)

20

Qui si guarda la percentuale di pacchetti persi. Ogni curva resta a zero finché la pipeline regge, poi sale: più traffico si manda oltre la capacità, più grande è la parte che non entra.

Esempio di calcolo. A circa 3 Gbit/s si mandano 5,8 milioni di pacchetti al secondo. P3 ne elabora 1,5 milioni. La parte persa è $1 - 1,5 / 5,8 \approx 74\%$. La baseline ne elabora 3,3 milioni: $1 - 3,3 / 5,8 \approx 43\%$.

Più la pipeline è costosa, più perde a parità di traffico: P3 (74%) > P2 (68%) > P1.5 (51%) > P1 (50%) > baseline (43%) > rxonly (21%).

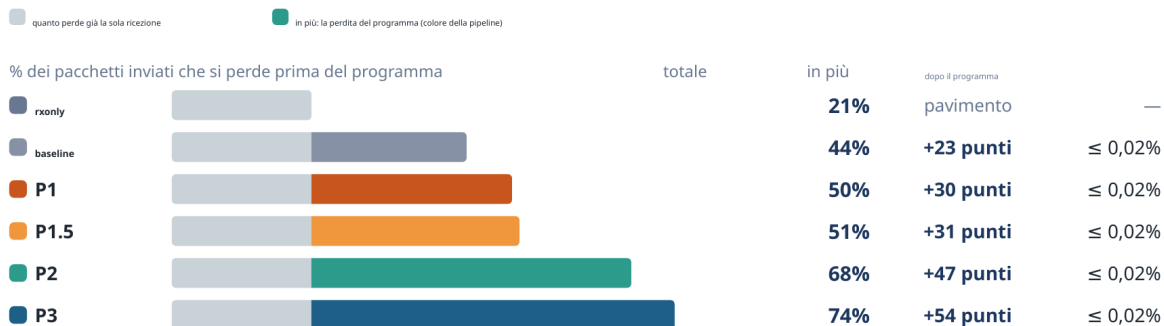
La linea rossa in basso è la perdita dopo il programma: resta a zero per tutte le pipeline e a ogni traffico (lo "0,02%" è rumore, vedi slide 19). Quindi il collo di bottiglia è il programma, non la trasmissione.

È il programma o la sola ricezione?

5 · LA DOMANDA DEL RELATORE · IL PROGRAMMA O LA RICEZIONE?

Oltre la sola ricezione, la perdita è del programma

A circa 3 Gbit/s anche il solo contatore (rxonly) perde il 21%: è il **pavimento**.



QUINDI

La perdita in più cresce con il **costo del programma**. Fino a 2,4 Gbit/s la sola ricezione non perde, e dopo il programma non si perde niente.

Traffico più alto della scala: 5,8 M pacchetti/s (5,7 per rxonly) · punti = differenza fra le due percentuali

21

Anche rxonly, che non fa niente oltre a ricevere, a circa 3 Gbit/s perde il 21%: 5,7 milioni di pacchetti al secondo sono troppi anche solo da ricevere. Quel 21% è il **pavimento**: la parte di perdita che non dipende dalla pipeline.

Il pezzo colorato di ogni barra è la perdita **in più** rispetto al pavimento: quella dovuta al lavoro del programma.

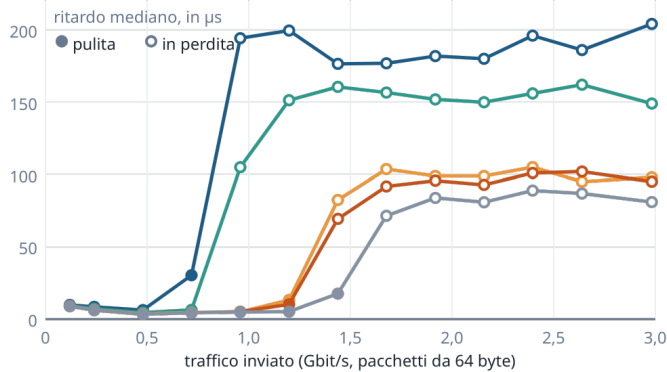
Esempio. La baseline perde il 44%, rxonly il 21%. La differenza, $44 - 21 = 23$ punti, è dovuta al programma. Per P3: $74 - 21 = 54$ punti. La perdita in più cresce con il costo del programma.

In più, fino a 2,4 Gbit/s rxonly non perde niente, mentre le pipeline cominciano a perdere molto prima (P2 e P3 già sotto 1 Gbit/s). Quindi nella zona in cui le pipeline cominciano a perdere, la ricezione da sola ce la fa ancora: il limite è davvero il programma.

Il ritardo

5 · LA DOMANDA DEL RELATORE · IL RITARDO

Il ritardo salta dove comincia la perdita



La coda piena spiega il ritardo

attesa prevista = 256 posti + capacità della pipeline

pipeline prevista misurata (p50)

baseline	77 μs	84 μs
P1	87 μs	95 μs
P1.5	90 μs	99 μs
P2	135 μs	154 μs
P3	171 μs	186 μs

Il resto (circa il 10%) è il percorso fino al nodo successivo.

QUINDI

Oltre la sua capacità il nodo non riesce a svuotare la coda, che resta **piena**: il ritardo si ferma e cresce la perdita.

Ritardo dal timbro del generatore all'arrivo al nodo successivo · mediana (p50) oltre la saturazione · capacità come nella slide 18

22

Il ritardo è il tempo dal momento in cui il generatore crea il pacchetto a quando arriva al nodo successivo. Il grafico mostra la mediana (il valore "tipico").

- Finché il nodo regge (pallini pieni) il ritardo è di pochi microsecondi: la coda è quasi vuota e ogni pacchetto viene elaborato subito.
- Quando comincia la perdita (pallini vuoti) il ritardo salta di colpo a 80–200 μs : la coda è piena, e ogni pacchetto deve aspettare che passino i 255 davanti a lui.
- Poi resta piatto: la coda non può diventare più lunga di 256 posti. Il traffico in più non aumenta l'attesa, viene semplicemente respinto.

Esempio di calcolo. P2 elabora 1,89 milioni di pacchetti al secondo, cioè uno ogni 529 ns. Con la coda piena un pacchetto aspetta $256 \times 529 \text{ ns} \approx 135 \mu\text{s}$. La misura dà 154 μs : il resto (circa il 10%) è il viaggio fino al nodo successivo.

La tabella a destra fa lo stesso conto per tutte le pipeline: previsto e misurato sono sempre vicini, quindi il salto del ritardo è proprio la coda piena.

Con due core la capacità raddoppia

5 · LA DOMANDA DEL RELATORE · PIÙ CORE

Con due core la capacità raddoppia

Capacità massima, in milioni di pacchetti al secondo



Un contatore per ogni core

Le statistiche sono per-core: ogni core incrementa la sua copia, senza istruzioni atomiche e senza contendersi una riga di cache. Chi legge somma.

Perché rxonly cresce meno

Tre thread su due code: due scrivono nella stessa coda e si intralciano (slide 13).

Una coda e un core, per scelta

Il banco misura il costo per core. Una scheda vera distribuisce i pacchetti su più code e più core (RSS).

QUINDI

Il costo per pacchetto resta quello di un core: un nodo con N code regge circa **N volte** la capacità misurata su un core, finché generatore e scheda reggono.

pktgen a 3 thread, 64 byte, 3 giri · due code d'ingresso, una per core (CPU 6 e 8) · rapporto = 2 core / 1 core

23

Finora il nodo usava un solo core e una sola coda, apposta, per misurare il costo per core. Qui si danno al nodo due core, ognuno con la sua coda: le pipeline elaborano quasi il doppio (da 1,91 a 1,98 volte).

Esempio. P3 passa da 1,49 a 2,95 milioni di pacchetti al secondo (×1,98). La baseline da 3,40 a 6,50 (×1,91).

Funziona perché i due core non si ostacolano: i contatori della pipeline sono uno per core. Se fossero un contatore unico, i due core dovrebbero passarselo a ogni pacchetto e si rallenterebbero a vicenda.

rxonly cresce meno (×1,62). La spiegazione più probabile è la coda condivisa: tre thread pktgen su due code, quindi due scrivono nella stessa coda e si intralciano (slide 13), e il danno pesa soprattutto su chi fa poco lavoro. Con pktgen su due core non è stato misurato a parte.

Conclusione: una scheda di rete vera distribuisce i pacchetti su più code e più core, con un solo scrittore per coda (la scheda); un nodo con N core regge circa N volte la capacità di un core, mentre il costo per pacchetto resta quello misurato.

In sintesi

IN CHIUSURA · SINTESI

In sintesi

1

Il dual boot rende le misure ripetibili

La variazione del costo scende da 15-160% a 0,3-0,6%. Il core del nodo non dorme, non cambia frequenza e nessuno lo interrompe.

2

Due generatori, due usi

pktgen confronta tutte le pipeline, da 3,31 a 1,50 M/s. xdp_gen, un thread per coda, misura P1-P3 senza la copia: 116-428 ns.

3

Si perde prima del programma

Il nodo satura alla sua capacità, il ritardo salta di un ordine di grandezza e la perdita è tutta all'ingresso: il collo di bottiglia è il programma, e la capacità cresce con i core.

Ettore Cantile · settembre 2026

25

- 1. Il dual boot rende le misure ripetibili.** Nella macchina virtuale lo stesso test dava risultati molto diversi fra una volta e l'altra; ora le ripetizioni coincidono, e si vedono anche differenze piccole come P1 contro P1.5.
- 2. Due generatori, due usi.** pktgen serve a confrontare tutte le pipeline (da 3,31 a 1,50 milioni di pacchetti al secondo). xdp_gen, con un thread per coda, dà il costo senza la copia per P1-P3 (116, 125, 293 e 428 ns di CPU); sopra la baseline la rete neurale costa uguale con i due.
- 3. Si perde prima del programma.** Quando il traffico supera la capacità, la coda si riempie, il ritardo salta e i pacchetti vengono respinti all'ingresso; dopo il programma non se ne perde nessuno. Il collo di bottiglia è il programma, e la capacità cresce con il numero di core.

Cose da sistemare o da dire con cautela. Slide 8 e 24: la variazione "0,5%" non si confronta direttamente con quella della macchina virtuale (slide 8). Slide 19, 20 e 21: " $\leq 0,02\%$ " è rumore di misura, meglio dire "zero".

Risolve il 30 settembre. Slide 10, 11, 13, 14, 15, 23 e 24: il "tetto del generatore" non era del generatore ma della coda condivisa; le cifre di xdp_gen sono ora con un thread per coda e in tempo di CPU. Slide 18: la riga P3 è corretta (0,72 / 0,96).